

PYTHON PROGRAMMING – COMPLETE EASY NOTES

Python is a high-level, object-oriented programming language.

It is a general-purpose language developed by Guido van Rossum in 1989.

Python is used for:

- ✓ Web Development
- ✓ Data Science
- ✓ Artificial Intelligence
- ✓ Automation

📌 Python = Simple + Powerful Programming Language

INPUT AND OUTPUT STATEMENTS

OUTPUT STATEMENT

Used to display output on the screen.

Syntax:

```
print("Message")
```

Example:

```
print("Hello Python")
```

Output:

Hello Python

INPUT STATEMENT

Used to take input from the user.

Syntax:

```
variable = datatype(input("Message"))
```

Example:

```
name = input("Enter Name: ")
```

📌 input() = Takes data from user

📌 print() = Displays data on screen

DATATYPES (VERY IMPORTANT)

Datatypes define the type of data stored in a variable.

Python commonly uses:

Datatype Meaning

int	Integer value
float	Decimal value
double	Large decimal value
char	Single character
list	Collection of values
tuple	Ordered collection
set	Unordered unique values
dictionary	Key-value pairs

COMMENTS

Comments are used to explain code.

SINGLE LINE COMMENT

Syntax:

```
# This is a comment
```

Example:

```
# Print statement
```

```
print("Hello")
```

📌 Comments are ignored by Python interpreter.

KEYWORDS

Keywords are reserved words that have special meaning in Python.

Python contains around 33 keywords (version dependent).

Examples:

if
else
for
while
class
def
try

BASIC SYNTAX

VARIABLE DECLARATION

Syntax:

```
variable_name = value
```

Example:

```
age = 20
```

LIST DECLARATION

Syntax:

```
list_name = [values]
```

Example:

```
numbers = [1,2,3]
```

TUPLE DECLARATION

Syntax:

```
tuple_name = (values)
```

Example:

```
data = (10,20,30)
```

SET DECLARATION

Syntax:

```
set_name = {values}
```

Example:

```
items = {1,2,3}
```

DICTIONARY DECLARATION

Syntax:

```
dictionary_name = {"key": "value"}
```

Example:

```
student = {"name": "Rahul"}
```

ARRAY DECLARATION

Syntax:

```
array_name = numpy.array([values])
```

Example:

```
arr = numpy.array([1,2,3])
```

STRING DECLARATION

Syntax:

```
string_name = "value"
```

Example:

```
name = "Python"
```

★ SIMPLE WAY TO REMEMBER

👉 List = []

👉 Tuple = ()

👉 Set = { }

👉 Dictionary = {Key : Value}

CONDITIONAL STATEMENTS

Used to make decisions according to conditions.

IF STATEMENT

Syntax:

if condition:

 statements

Example:

if age >= 18:

 print("Adult")

IF ELSE STATEMENT

Syntax:

if condition:

 statements

else:

 statements

Example:

if age >= 18:

 print("Adult")

else:

 print("Minor")

NESTED IF STATEMENT

Used when one if statement is inside another if statement.

Syntax:

if condition1:

```
    if condition2:
        statements
    else:
        statements
else:
    statements
```

ELSE IF STATEMENT

Used to check multiple conditions.

Syntax:

```
if condition1:
    statements
elif condition2:
    statements
else:
    statements
```

LOOPING STATEMENTS

Loops are used to execute code repeatedly.

WHILE LOOP

Syntax:

```
while condition:
    statements
    increment/decrement
```

Example:

```
while i < 5:
    print(i)
```

```
i += 1
```

FOR LOOP

Syntax:

```
for variable in range(start,end):  
    statements
```

Example:

```
for i in range(1,5):  
    print(i)
```

★ **SIMPLE WAY TO REMEMBER LOOPS**

👉 for loop = Fixed number of repetitions

👉 while loop = Runs until condition becomes false

FUNCTIONS (VERY IMPORTANT)

Functions are reusable blocks of code.

FUNCTION DECLARATION

Syntax:

```
def function_name():  
    statements
```

Example:

```
def hello():  
    print("Hello")
```

FUNCTION CALLING

Syntax:

```
function_name()
```

Example:

hello()

OBJECT ORIENTED PROGRAMMING (OOP)

OOP is a programming method based on objects and classes.

Main concepts:

- ✓ Class
 - ✓ Object
 - ✓ Encapsulation
 - ✓ Abstraction
 - ✓ Inheritance
 - ✓ Polymorphism
-

CLASS

Class is a blueprint for creating objects.

Syntax:

```
class class_name:  
    data_members  
    methods
```

Example:

```
class Student:  
    pass
```

OBJECT

Object is an instance of a class.

Syntax:

```
object_name = class_name()
```

Example:

```
student1 = Student()
```

Calling function:

object_name.function_name()

ENCAPSULATION

Encapsulation means binding data and functions into a single unit.

Example:

class Student:

 data_members

 member_functions

👉 Class provides encapsulation.

ABSTRACTION

Abstraction means hiding low-level details from users.

Access levels:

__private

_protected

public

👉 Only required information is shown to users.

INHERITANCE

Inheritance means creating a new class from an existing class.

Parent Class:

class Parent:

 properties

Child Class:

class Child(Parent):

 properties

Types of inheritance:

- ✓ Single
 - ✓ Multiple
 - ✓ Multilevel
 - ✓ Hierarchical
 - ✓ Hybrid
-

POLYMORPHISM

Polymorphism means showing different behavior in different situations.

Python does not directly support function overloading.

METHOD OVERRIDING

Same function name in parent and child classes.

Example:

```
class Parent:
```

```
    def fun():
```

```
        pass
```

```
class Child(Parent):
```

```
    def fun():
```

```
        pass
```

OPERATORS

Operators are used to perform operations on values.

ARITHMETIC OPERATORS

Operator Meaning

- | | |
|---|----------------|
| + | Addition |
| - | Subtraction |
| * | Multiplication |

Operator Meaning

/ Division

% Modulus

RELATIONAL OPERATORS

Operator Meaning

> Greater than

< Less than

>= Greater equal

<= Less equal

== Equal

!= Not equal

LOGICAL OPERATORS

Operator Meaning

and Logical AND

or Logical OR

not Logical NOT

ASSIGNMENT OPERATORS

Used to assign values.

Example:

=

TERNARY OPERATOR

Used for short if-else statements.

Syntax:

value1 if condition else value2

EXCEPTION HANDLING (VERY IMPORTANT)

Exception handling is used to handle errors during program execution.

Syntax:

try:

 exception code

except ExceptionName:

 message

finally:

 always execute

Example:

try:

 print(10/0)

except:

 print("Error")

finally:

 print("Completed")

FINAL SUMMARY

✓ Python = High-level programming language

✓ print() = Display output

✓ input() = Take user input

- ✓ Datatypes = Store different types of data
- ✓ Conditional statements = Decision making
- ✓ Loops = Repeat tasks
- ✓ Functions = Reusable code
- ✓ Class & Object = OOP concepts
- ✓ Inheritance = Reuse existing code
- ✓ Polymorphism = Different behaviors
- ✓ Exception Handling = Error management

📌 Python is widely used in Web Development, AI, Data Science, Automation, and Software Development.